PROJECT REPORT: Interactive Wall Visualization System

Module 1: Asset Management, Design Logic & Database Architecture

Role: Person 1 (Database & Asset Architect) **Project Segment:** Backend Data Engineering & Resource Allocation

PAGE 1: EXECUTIVE SUMMARY & OBJECTIVES

1.1 Project Overview

The Interactive Wall Visualization and Redesigning System is a dynamic 2D software application designed to allow users to digitally redesign interior spaces. By utilizing advanced image processing, matrix transformations, and a robust User Interface (UI), the application provides a real-time preview of wall customizations, including doors, windows, textures, and solid paint colors.

1.2 Role of Person 1 (Design Database)

In a modular software team, separation of concerns is critical. Person 1 is exclusively responsible for the **Data Access Layer**. The primary objective is to create a robust, error-proof, and highly organized asset library. This module acts as the central repository from which the UI Team (Person 6) and the Image Processing Team (Persons 2 & 4) retrieve assets.

By centralizing the data, we ensure that:

- Image processors do not need to hardcode file paths.
- The UI team does not need to guess asset names or IDs.
- The system footprint remains lightweight and memory-efficient.

PAGE 2: DIRECTORY ARCHITECTURE & PATHING

2.1 The Cross-Platform Challenge

A common point of failure in collaborative software engineering is the "It works on my machine" syndrome. If file paths are hardcoded using Absolute Pathing (e.g.,

`C:/Users/Person1/Documents/Project/assets/door.png`), the application will immediately crash when transferred to Person 6, as their local directory structure is entirely different.

2.2 The Relative Path Solution

To guarantee seamless portability, this database strictly utilizes **Relative Pathing**.

Plaintext
```
📁 Interactive_Wall_Project/          [Root Directory]
 │
 ├── 📄 design_database.py            [Data Retrieval Logic]
 ├── 📄 test_design_database.py       [Validation Suite]
 │
 └── 📁 assets/                       [Master Media Repository]
     ├── 📁 bookshelf/                (14 variations)
     ├── 📁 decorations/              (14 variations: JPG/PNG)
     ├── 📁 doors/                    (12 variations)
     ├── 📁 lights/                   (8 variations)
     ├── 📁 switchboards/             (8 variations)
     ├── 📁 tables/                   (12 variations)
     ├── 📁 Textures/                 (7 variations)
     ├── 📁 wallpapers/               (5 variations)
     └── 📁 windows/                  (16 variations)
```

By placing `design_database.py` parallel to the `assets` folder, the Python script uses the relative path string `"assets/doors/wooden_door.png"`. This instructs the interpreter to look inside the current working directory, allowing the entire project folder to be zipped, emailed, and executed flawlessly on any Windows, macOS, or Linux machine.

PAGE 3: DATA STRUCTURES & CODE ANALYSIS

3.1 Big-O Optimization: The Nested Dictionary

To ensure lightning-fast retrieval times, the asset library is constructed using a nested Python Dictionary (`{}`).

Python
```
DESIGNS = {
    "door": [
        {"id": "door_clasic_1", "name": "Classic Wooden Door 1", "path":
"assets/doors/wooden_clasic_door_1.png"},
```

```
        # ... additional door assets
    ],
    "window": [
        # ... window assets
    ]
}
```

Technical Justification:

- **O(1) Time Complexity:** Dictionaries in Python use hash maps. When Person 6 requests `get_designs("door")`, the system does not iterate through windows, tables, or lights. It executes an O(1) instant lookup directly to the door array, ensuring the UI remains highly responsive even if the database scales to thousands of items.
- **Metadata Encapsulation:** Each individual design is stored as an inner dictionary containing three vital pieces of metadata:
    1. `id`: A stable, immutable identifier used by the backend matrix logic.
    2. `name`: A human-readable string passed to Person 6 for UI dropdown menus.
    3. `path`: The system location passed to Person 4 for image rendering.

3.2 Graceful Failure Mechanisms

Software must anticipate human error. If the UI module accidentally requests an object category that does not exist, standard dictionary lookups (`DESIGNS["ceiling_fan"]`) would trigger a fatal `KeyError` and crash the application.

The Implementation:

Python
```
def get_designs(object_type: str) -> list:
    type_map = {
        "doors": "door", "door": "door",
        "windows": "window", "window": "window"
    }
    normalized_type = type_map.get(object_type.lower(), object_type.lower())
    return DESIGNS.get(normalized_type, [])
```

Technical Justification:

- **Input Normalization:** The `.lower()` method prevents crashes from varying capitalization (e.g., "DOOR" vs "door"). The `type_map` dictionary automatically catches and corrects pluralization (converting "doors" to "door").

- **Safe Return (`.get`):** By utilizing `.get(normalized_type, [])`, the system safely returns an empty list if a key is not found, allowing the UI to handle the missing data gracefully without crashing the core execution thread.

PAGE 4: MEMORY OPTIMIZATION & RGB LOGIC

4.1 The Memory Deficit of Image-Based Colors

A critical requirement of the Interactive Wall System is allowing users to "paint" the wall. A rudimentary approach would involve generating and saving flat `.png` image files for every possible color (e.g., `blue_wall.png`, `red_wall.png`).

This approach is fundamentally flawed:

1. **Bloated File Size:** High-resolution image files consume megabytes of disk space.
2. **RAM Exhaustion:** Loading multiple full-screen images into active memory strains the system hardware.
3. **Lack of Scalability:** Adding 100 new colors would require generating 100 new image files.

4.2 The Mathematical Solution: 8-Bit RGB Tuples

To solve this, Person 1 implemented a purely mathematical color repository.

Python
```
WALL_COLORS = {
   "Pure White": (255, 255, 255),
   "Sage Green": (188, 198, 163),
   "Charcoal": (54, 69, 79),
   "Navy Blue": (0, 0, 128)
}
```

Technical Justification:

- **Direct Hardware Rendering:** Colors are defined by Red, Green, and Blue light values, scaled from `0` to `255` (8-bit color depth). The tuple `(188, 198, 163)` takes mere bytes of memory. Person 2 (Image Processing) can apply these exact mathematical coordinates directly to the wall matrix, rendering the color instantly.
- **Immutability:** The data is stored in Python Tuples `()`. Tuples are immutable, ensuring that no other module can accidentally overwrite or alter the core color definitions during runtime.

PAGE 5: VALIDATION SUITE & AUTOMATED TESTING

5.1 The Automated Testing Philosophy

Before hand-off, the database must be rigorously tested to prevent runtime errors. The `test_design_database.py` script serves as an automated QA (Quality Assurance) module.

5.2 Operating System Level Verification

Python

```
for asset in all_assets:
    if not os.path.exists(asset["path"]):
        print(f"⚠ MISSING FILE: {asset['path']}")
```

Rather than merely checking if the strings are formatted correctly, the script utilizes the `os` library to interface directly with the host computer's operating system. It physically traverses the hard drive to verify the presence of every single `.png` and `.jpg`. If a file name contains a typo, the test will flag it.

5.3 Mathematical Boundary Validation

Python

```
for color_name, rgb in colors.items():
    if len(rgb) != 3 or not all(0 <= val <= 255 for val in rgb):
        print(f"⚠ INVALID RGB FOR '{color_name}': {rgb}")
```

Image processors like OpenCV will crash if they receive an RGB value outside the standard 8-bit range. The test suite dynamically loops through every color in the database, guaranteeing that the tuple contains exactly three integers, and that every integer is mathematically bounded between `0` and `255`.

PAGE 6: UI TEAM INTEGRATION MANUAL (PERSON 6)

6.1 Handoff Summary

The database is fully stable, structurally sound, and tested. Person 6 does not need to parse folders or write file paths.

6.2 Implementation Snippets

To populate the Streamlit User Interface, Person 6 will utilize the following import logic:

Python

```
# 1. Import the necessary data access functions
from design_database import get_designs, get_wall_colours

# 2. Populating a Dropdown Menu for Windows:
available_windows = get_designs("window")
for window in available_windows:
    print(f"Adding {window['name']} to dropdown menu...")
    # window['path'] is held in memory for Person 4 to use upon user click

# 3. Applying a Solid Paint Color:
wall_palette = get_wall_colours()
selected_color = wall_palette.get("Navy Blue")
# selected_color now perfectly holds the tuple (0, 0, 128)
```